TileWorld Multi-Agent System

AI6125 Multi-Agent Systems
Group 18
and Ananya Singh Twissa Modi Aditya Kumar Goel
Yu Taek Lee Nik Shahrul Nafiz Bin Nik Azhan Máte Tringer

Abstract

We present a six-agent multi-agent system built for the TileWorld simulation using the MASON framework. TileWorld is a challenging benchmark in which agents must explore a partially observable grid, discover a hidden fuel station, collect tiles, and fill holes to earn reward — all under tight fuel constraints and with objects that expire after a limited lifetime. Our approach uses a two-layer architecture: a shared infrastructure layer that provides A* pathfinding, fuel-aware planning, memory with observation timestamps, and a broadcast communication protocol, plus a specialisation layer of six role-specific agents built on top of it. Through four development phases, average reward on Config 1 improved from 32.1 to 864.3. A 10-run Phase 4 validation on Config 2 achieved an average reward of 888.6 with zero failures, though Config 2 carries structural reliability risk not fully captured by a small sample. The largest single gain came from introducing multi-agent coordination in Phase 3, while Phase 4 added role-specific behaviour grounded in the bold-vs-cautious agent framework of Pollack and Ringuette (1990).

Introduction

TileWorld is a well-known testbed for evaluating agent architectures under partial observability and resource constraints (1). Six agents share a bounded grid and must cooperate to maximise the number of tiles successfully delivered into holes within a fixed simulation budget. The environment presents several concurrent challenges: agents can only see a small local area, so global map knowledge must be built incrementally through movement and communication; fuel is consumed on every move and can only be replenished at a single hidden station; and both tiles and holes expire after a limited number of steps, making timing and freshness critical.

A naive solution — six independent agents each running the same generic loop — wastes effort through redundant exploration, duplicate targeting, and uninformed routing. Our goal was to design a system that behaves as a coordinated team while keeping each agent's design tractable and individually testable.

We address this with a *two-layer architecture*. The first layer is shared infrastructure implemented in three classes: `SmartTWAgent` (the base agent), `SmartTWAgentMemory` (time-stamped shared memory), and `SmartTWPlanner` (goal-directed, fuel-aware planning). Every improvement to this layer benefits all six agents simultaneously. The second layer is a set of specialist agents that extend the base class and each prioritise a different aspect of the task: fuel discovery, tile collection, delivery reliability, map coverage, high-value routing, and failure prevention.

The rest of this paper is organised as follows. Section 2 places the work in the context of TileWorld research. Section 3 describes the shared architecture. Section 4 covers the communication protocol. Section 5 presents each specialist agent. Section 6 describes the experimental configurations. Section 7 reports results. Section 8 analyses strengths and weaknesses, and Section 9 concludes.

Background

TileWorld was introduced by Pollack and Ringuette (1) as an experimental platform for studying agent deliberation. Their central concern was the trade-off between *bold* agents that commit to plans and execute them without interruption, and *cautious* agents that reconsider their plan at every step. They found that neither extreme dominates: bold agents waste effort reconsidering, while cautious agents miss opportunities by abandoning plans too eagerly. The optimal balance depends on the environment's rate of change.

Subsequent work surveyed in (2) confirmed that the right level of commitment varies with environmental volatility. High-volatility settings (many objects, short lifetimes) reward cautious behaviour; low-volatility settings (sparse, long-lived objects) reward bold commitment.

Our architecture operationalises this insight directly. The shared planner uses different affordability margins depending on whether the runtime environment is detected as short-lifetime or stable. The `SmarterReplanningAgent` specialist functions as a con-

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

figurable cautious agent: in volatile conditions it projects whether a target will survive until arrival and abandons the plan proactively if not; in stable conditions it uses a looser threshold and commits longer.

System Architecture

Two-Layer Design

The architecture separates concerns into two layers.

Layer 1 — Shared Foundation. All six agents inherit from `SmartTWAgent` and use the same `SmartTWAgentMemory` and `SmartTWPlanner`. This layer provides: A* pathfinding with obstacle avoidance and fuel awareness; a memory store with per-cell observation timestamps and automatic staleness tracking; a claim system to prevent two agents targeting the same object; runtime environment classification (`isDense()`, `isShortLifetime()`, `isLargeGrid()`); and a full broadcast communication stack.

Layer 2 — Specialist Agents. Each specialist extends `SmartTWAgent` and overrides the `think()` method to insert role-specific decisions before delegating to the shared base logic via `super.think()`. This keeps each specialist focused on its role while retaining all base capabilities unchanged.

Shared Agent Loop

Every simulation step follows a fixed ordering enforced by the MASON scheduler:

1. **Environment step** (ordering 1): new objects are created, expired objects are removed, and the message list is cleared.
2. **Sense and communicate** (ordering 2): each agent observes nearby cells, updates memory, reads incoming messages, and broadcasts its own.
3. **Think and act** (ordering 3): each agent runs its decision logic and executes one action (move, pickup, drop, refuel, or wait).

Within ordering 2, agents run `sense(); communicate();` sequentially in creation order, each reading the message queue before appending to it. A message broadcast by agent N is therefore visible during the same step to agents $N{+}1$ through 6 (which read after N writes), but not to agents 1 through $N{-}1$ (which already read before N wrote). By ordering 3, all broadcasts from ordering 2 are in the queue; however, each agent's stored parsed data reflects only the messages that existed when it called `communicate()`, so earlier-ordered agents see later-ordered agents' broadcasts at the following step rather than the current one. In practice this means fuel station discoveries and tile or hole sightings propagate to the full team by the following step at the latest, and to later-ordered agents within the same step.

A* Pathfinding

Navigation is handled by `AstarPathGenerator`, which computes obstacle-avoiding paths using the standard A* algorithm with Manhattan distance as the heuristic. The planner wraps path generation with an affordability check: before committing to a goal, it verifies that the agent has enough fuel to reach the target and return to the fuel station. In short-lifetime environments the planner also rejects targets whose estimated remaining lifetime is shorter than the steps required to arrive.

Memory and Environment Detection

`SmartTWAgentMemory` stores observation timestamps for every seen cell, tile, and hole. These timestamps drive three runtime signals:

- `isShortLifetime()` — true when observed object disappearance rates suggest objects expire in roughly 30 steps or fewer.
- `isDense()` — true when the observed object creation rate is high.
- `isLargeGrid()` — true when the map appears to exceed roughly 60 cells on a side.

These signals are estimated from observations rather than hardcoded to a specific configuration, so the same agents adapt correctly to both Config 1 and Config 2 without any parameter changes.

Communication Protocol

Agents communicate via a global broadcast channel. Because ordering 2 runs agents sequentially, a broadcast from agent N reaches later-ordered agents within the same step and earlier-ordered agents by the following step at the latest; the full team has the message within one step. The base class parses all incoming messages and stores results in shared protected fields, so specialist agents consume parsed data directly without writing any parsing code themselves.

Table 1 lists the nine message types.

Type	Format	Purpose
`FUEL`	`x,y`	Fuel station location
`TILE`	`x,y,time`	New tile sighting
`HOLE`	`x,y,time`	New hole sighting
`GONE`	`x,y`	Object disappeared
`CLAIM`	`x,y`	Agent is targeting this cell
`LOW`	`agentId,fuel`	Low-fuel warning
`EXPIRING`	`x,y,reason`	Target likely to expire
`HOTSPOT`	`x,y,density`	High-density area
`SWAP`	`agentId,status`	Zone exploration complete

Table 1: The nine message types and their payloads.

Communication improves team performance in four ways. First, **faster discovery sharing**: fuel station discovery by any one agent immediately benefits all five others. Second, **deduplication**: CLAIM messages prevent two agents from wasting movement on the same target. Third, **freshness control**: EXPIRING warnings let all agents avoid routing toward targets that are about to vanish. Fourth, **coverage coordination**: HOTSPOT

and SWAP messages guide exploration toward productive and uncovered regions.

Specialist Agents

Each specialist inherits all base capabilities and adds a focused decision layer in `think()`.

FuelScoutAgent

`FuelScoutAgent` is designed to accelerate fuel station discovery. While the station is unknown, it uses a search radius of 20 cells — significantly wider than the base agent's lawnmower pattern — to jump between zones and cover the map faster. Its refuel threshold adapts to grid size (`isLargeGrid() ? 250 : 175`) to ensure it never runs critically low before the station is found. Once fuel is known and broadcast, it reverts to standard base behaviour. It also sends `LOW` warnings when fuel drops below 150, alerting teammates to shift coverage away from fuel-scarce regions.

TileHunterAgent

`TileHunterAgent` focuses on efficient tile collection. Its key behaviour is adaptive batching: in stable, long-lifetime environments it batches up to three tiles before delivering, amortising the fuel cost of a delivery trip over three reward points; in short-lifetime environments it delivers after every single tile to avoid objects expiring in transit. It immediately claims any tile it targets to reduce collisions and uses `HOTSPOT` data to bias movement toward tile-dense regions.

HoleFillerAgent

`HoleFillerAgent` protects the team from wasted delivery trips. When carrying a tile it checks whether the nearest hole has been flagged as expiring by a teammate; if so, it voids the current plan and seeks an alternative. In dense environments it patrols within five cells of a known hole while empty, reducing the distance it must travel once it acquires a tile. In sparse environments it skips patrol entirely, since the fuel cost of circling outweighs the benefit. It also rebroadcasts fresh hole locations at intervals to keep the team's delivery knowledge current.

ExplorerAgent

`ExplorerAgent` ensures systematic map coverage. It uses `getLeastVisitedTarget()` with a search radius that adapts to grid size (`isLargeGrid() ? 20 : 10`) to always move toward the least-recently-seen area in its assigned zone. It biases movement toward `HOTSPOT` positions broadcast by the `DeliveryOptimizerAgent`, but only when the hotspot lies within its own zone. When coverage exceeds 80% it broadcasts a `SWAP` message to signal zone completion, allowing the team to shift exploration capacity elsewhere.

DeliveryOptimizerAgent

`DeliveryOptimizerAgent` maximises the reward per fuel unit. When carrying a tile, it selects the target hole by scoring all known holes on three factors: distance from current position, freshness (age relative to lifetime), and proximity to `HOTSPOT` regions where additional holes and tiles are likely to appear. This multi-gate scoring avoids the pathological cases of pure nearest-hole routing (choosing a hole that is about to expire) and pure freshness routing (choosing a distant hole over a nearby one). It also broadcasts `HOTSPOT` messages from high-density areas it observes, sharing information that directly improves the Explorer's coverage decisions.

SmarterReplanningAgent

`SmarterReplanningAgent` implements the adaptive cautious agent concept from Pollack and Ringuette (1). At each step it applies three checks on any active plan. First, it checks whether a teammate has already flagged the current goal as expiring; if so, it voids the plan immediately. Second, it independently projects whether the goal will still exist on arrival using observation timestamps and estimated remaining lifetime; the expiry threshold adapts to environment volatility (`isShortLifetime() ? 0.7 : 0.9`). Third, it checks whether the agent has enough fuel to reach the goal and return to the fuel station, with a safety margin that scales with grid size (`max(50, gridWidth / 4)`). Failed checks trigger early replanning rather than waiting for the plan to fail at execution time, and the agent broadcasts an `EXPIRING` message to warn teammates of bad targets it detects.

Experimental Configurations

The system was evaluated on two configurations derived from the MASON TileWorld parameters. Table 2 summarises their key differences.

Parameter	**Config 1**	**Config 2**
Grid size	50×50	80×80
Tile rate (per step)	0.2	2.0
Hole rate (per step)	0.2	2.0
Obstacle rate (per step)	0.2	2.0
Object lifetime (steps)	100	30
Agent fuel	500	500

Table 2: Parameters for Config 1 and Config 2.

Config 1 is the easier environment: objects are sparse but long-lived, so careful planning and commitment pay off. Config 2 is harder: the larger grid puts more pressure on fuel, and the short object lifetime means memory information becomes stale quickly, increasing the cost of a bad plan.

Results

Phase Development (Config 1)

Table 3 shows the average reward achieved at each development phase, measured on Config 1. Each phase added a distinct capability.

Phase	Avg	Min	Max	Key addition
Phase 1	32.1	0	55	Survival and memory
Phase 2	160.3	0	249	A* planning, delivery
Phase 3	541.5	469	601	Coordination, communication
Phase 4	864.3	801	959	Adaptive base + role specialisation

Table 3: Config 1 average reward by development phase. Phases 1–3 are frozen historical measurements from earlier codebase states; Phase 4 is a current rerun on the active codebase after all follow-up commits.

Phase 1 established a survival baseline: `SmartTWAgent` and `SmartTWAgentMemory` replaced the default agent with fuel-aware movement, memory decay, and opportunistic tile pickup. The system reacted to local events but could not yet route toward high-value work.

Phase 2 added `SmartTWPlanner` with A* pathfinding. Agents stopped wandering and began executing goal-directed plans. The first major reward jump followed because the same map knowledge was now exploited efficiently.

Phase 3 introduced multi-agent coordination: zone-based exploration, message broadcasts, shared tile and hole knowledge, and claim-based deconfliction. The score jump from Phase 2 to Phase 3 (160.3 to 541.5, a 3.4× increase) was the largest in the project and came from structural rather than incremental improvement: six agents began working as a team rather than as independent individuals.

Phase 4 unfolded in three steps. First, adaptive improvements to `SmartTWAgent` and `SmartTWAgentMemory` directly: runtime environment detection, adaptive fuel safety margins, and smarter planning heuristics. Second, a two-stage architectural transition: the uniform agent loop was first broken into individually instantiated person-named subclasses (one per team member), then renamed to role-named classes (`TileHunterAgent`, `HoleFillerAgent`, etc.), both stages initially delegating entirely to the shared base. Third, follow-up commits added the substantive role-specific logic — tighter expiry thresholds, freshness-aware delivery scoring, reachability checks, and throttled broadcasts. The 864.3 average reflects this full three-step progression; the largest gains came from the adaptive base improvements and the coordination already established in Phase 3, with role-specific behaviour providing targeted quality refinements on top.

Final Validation (Phase 4)

Table 4 shows the Phase 4 validation results on both configurations.

Config	Avg reward	Final reward	Failures
Config 1	864.3	959	0
Config 2	888.6	981	0

Table 4: Phase 4 validation results. Each config was evaluated with `TileworldMain` running 10 independent random-seed trials (`iteration = 10`; see `TileworldMain.java:30`). Average reward is the mean final score across the 10 trials; Final reward is the score of the last trial. A failure is a trial whose final reward was zero. All three aggregate metrics are recorded in `MULTI_RUN_RESULTS.md`. Per-trial variance (individual min/max across the 10 trials) was not separately recorded for Config 2.

Both 10-run validations completed with zero failures. Config 2 returned a slightly higher average (888.6 vs. 864.3), but Config 1 remains the clearest success case because its environment is more amenable to the coordination and planning strategies the system relies on. Config 2 structural risks are discussed in the analysis below.

Analysis

Strengths

Shared-base scalability. Implementing pathfinding, memory, and communication once in the base class means every algorithmic improvement benefits all six agents simultaneously. This was the most important engineering decision in the project.

Coordination under partial observability. Without communication, each agent is limited to what it can directly observe. The broadcast protocol converts six local observers into a distributed shared-intelligence system. Claims prevent duplicated effort; EXPIRING warnings stop wasted movement; HOTSPOT messages direct exploration toward productive regions.

Measured performance trajectory. The phase scores confirm that each architectural addition produced a measurable gain. The largest gain (Phase 3 coordination) accounts for the majority of total improvement, validating the design priority of getting coordination right before tuning specialisation.

Cross-agent idea borrowing. Several mechanisms were refined across agents. For example, `DeliveryOptimizerAgent` explicitly combines expiry projection (originally developed for `SmarterReplanningAgent`), hotspot routing, and freshness-aware scoring into a single delivery policy. Good ideas spread through the team incrementally.

Weaknesses

Config 2 structural risk. The 10-run validation produced a positive average reward (888.6), but the environ-

ment's structure creates latent reliability risk not fully captured by the aggregate. The 80×80 grid creates a larger search space for the fuel station; delayed discovery forces the whole team into conservative routing. Short object lifetimes increase the chance that remembered targets expire before an agent arrives, even with predictive expiry checking. Per-trial variance (individual min/max across the 10 trials) was not separately recorded for Config 2, so the spread of outcomes is unquantified.

Early-run lifetime detection. The runtime environment classifier requires enough observed disappearances to estimate object lifetime accurately. In the first hundred steps of a Config 2 run, agents may still believe they are in a stable environment and apply overly generous planning horizons.

Fuel station discovery bottleneck. Many decisions across the system depend on the fuel station being found early. On a large map with an unlucky starting layout, delayed discovery creates cascading conservative behaviour across all agents until the station is located.

Coordination Tradeoff

The fundamental tradeoff in any multi-agent system is between the parallelism benefit — six agents covering the map simultaneously, collecting tiles, and filling holes in parallel — and the coordination cost of communication, claim management, and zone assignment overhead. This system is strongest when the environment gives coordination time to pay off: Config 1, with its stable, sparse objects, rewards careful planning and reliable delivery. Config 2 tests the limits because the environment changes fast enough that shared information can become stale before the team acts on it.

Conclusion

We presented a six-agent TileWorld multi-agent system that improves from a survival baseline (Phase 1, avg. 32.1) to a coordinated specialist team (Phase 4, avg. 864.3 on Config 1) through four development phases. A 10-run Phase 4 validation on Config 2 achieved an average reward of 888.6; the four-phase development history is documented for Config 1 only. The key design insight is the two-layer separation between shared infrastructure and role specialisation: improvements to the base layer compound across all agents, while specialist layers add targeted decision quality without duplicating infrastructure.

The most impactful single change was the introduction of multi-agent coordination in Phase 3, confirming the importance of communication and deconfliction over individual agent optimisation. Phase 4 role specialisation added further gains by covering different failure modes of the task: fuel discovery, tile collection efficiency, delivery reliability, map coverage, high-value routing, and predictive failure prevention.

Future work could improve Config 2 reliability through two main directions: earlier fuel station discovery via a dedicated fast-scout strategy that temporarily overrides zone assignments at the start of a run, and adaptive lifetime thresholds that update more aggressively in the first 50 steps based on early disappearance observations.

References

[1] Pollack, M. E., and Ringuette, M. 1990. Introducing the Tileworld: Experimentally evaluating agent architectures. In *Proceedings of the Eighth National Conference on Artificial Intelligence (AAAI-90)*, 183–189.

[2] Kinny, D., and Georgeff, M. 1991. Commitment and effectiveness of situated agents. In *Proceedings of the Twelfth International Joint Conference on Artificial Intelligence (IJCAI-91)*, 82–88.